

Autómatas, Teoría de Lenguajes y Compiladores

Proyecto Especial

Instituto Tecnológico de Buenos Aires



Grupo G104

Integrantes:

Lucas Di Candia (63212)

Agostino Alfieri (64607)

Francisco Tomas Mangiaterra (64544)

Nicolas Canzonieri (63501)

Fecha de Entrega: 28/11/2025

2025

Tabla de Contenidos

1. Introducción.....	3
2. Modelo Computacional.....	3
2.1 Dominio.....	3
2.2 Lenguaje.....	3
3. Implementación.....	5
3.1 Frontend.....	5
3.2 Backend.....	5
3.3 Dificultades Encontradas.....	5
4. Futuras Extensiones.....	6
5. Conclusiones.....	6
6. Bibliografía.....	6

1. Introducción

El presente trabajo consistió en el desarrollo de un compilador denominado Partitex, con el objetivo de permitir la escritura y generación de partituras musicales a partir de un lenguaje de entrada basado en texto plano, en el marco de la materia Autómatas, Teoría de Lenguajes y Compiladores. Se buscó principalmente brindar una opción ágil y legible al usuario para componer estructuras musicales sin depender de software de edición visual complejo.

El compilador fue desarrollado en el lenguaje de programación C. Una de las principales herramientas empleadas fue Flex para la definición y generación del analizador léxico, el cual se encarga de identificar las notas, figuras rítmicas y metadatos del lenguaje. Otra importante herramienta utilizada fue Bison, que se utilizó para construir el analizador sintáctico para validar que la estructura de la composición siga la gramática definida, además de construir las estructuras internas necesarias para la posterior generación del código de salida.

El proceso se separó en tres etapas. La primera consistió en la propuesta y diseño de la idea principal, donde se definieron los requerimientos de la notación musical textual. En la segunda etapa, se llevó adelante el desarrollo del frontend, construyendo el AST (Árbol de Sintaxis Abstracta) que representa la canción. Finalmente, la tercera etapa consistió en el terminado de validaciones semánticas (como la coherencia rítmica) y la generación del código final.

2. Modelo Computacional

El proceso de desarrollo del compilador se basó en tres etapas, como se mencionó anteriormente. En el proceso de diseño, se determinaron el dominio musical y el lenguaje de especificación.

2.1 Dominio

El dominio del compilador desarrollado en este proyecto se centró en la representación de notación musical melódica. Se diseñó un lenguaje de entrada centrado en la definición de notas, octavas, alteraciones (sostenidos/bemoles) y duraciones, así como metadatos de la obra (título, compás, tempo).

El enfoque estuvo en la validación de la estructura musical lineal. Esta decisión permitió reducir la complejidad del proyecto para centrarse en el correcto modelado de compases y figuras rítmicas. Se prioriza la capacidad de definir la música de manera secuencial, permitiendo al usuario "programar" su música de forma estructurada.

2.2 Lenguaje

El lenguaje de entrada es un formato de texto plano diseñado para ser intuitivo. A cada elemento de la teoría musical se lo mapeó con una sintaxis específica. Por ejemplo, la definición de una nota y su duración:

```
key do4 major;
```

```
time 4/4;
```

tempo allegro;

notes: do4 quarter, re4 quarter, mi4 quarter, fa4 quarter;

En este ejemplo se aprecia la legibilidad: se definen una sección de notas donde se especifica la nota (do, re, mi), la octava (4) y la duración (q = quarter/negra). Las comas “,” delimitan los compases.

Salida Esperada:

Key: C major

Time signature: 4/4

Tempo: allegro

Notes: C4/4, D4/4, E4/4, F4/4

MIDI Visualization:

```

C      #####-----
C#     -----
D      ----#####-----
D#     -----
E      -----#####-----
F      -----#####
F#     -----
G      -----
G#     -----
A      -----
A#     -----
B      -----

```

A nivel sintáctico, se implementaron las siguientes representaciones:

- Notas musicales y octava numérica.
- Figuras musicales (redonda, blanca, negra, corchea, etc.).
- Agrupación por compases explícitos.
- Repeticiones o secciones definidas.

El lenguaje de salida corresponde a un formato intermedio de representación textual simplificada que permite la visualización o posterior procesamiento de la partitura.

3. Implementación

Para la implementación del compilador se utilizó como base el proyecto Flex-Bison-Compiler, facilitando la integración de las herramientas de análisis léxico y sintáctico.

3.1 Frontend

Para desarrollar el frontend, el énfasis estuvo en la definición de una gramática formal que represente la estructura musical. Se definieron como símbolos terminales: identificadores de notas (C, D, E...), números (para octavas y tempos), y palabras reservadas (key, time, tempo, etc.). Como no terminales se definieron las estructuras de alto nivel:

- El header que contiene la configuración global.
- El body contiene la secuencia de notas o compases.
- El símbolo inicial program.

Se crearon estructuras en C para el AST, tales como NoteNode, MeasureNode y SongNode. Se buscó modularizar las acciones sintácticas para que, al detectar una nota o un silencio, se instancie el nodo correspondiente con sus atributos (frecuencia, duración) y se adjunte a la lista del compás actual.

3.2 Backend

La etapa de backend se centró en la validación semántica y la generación de código. Dado que la música tiene reglas matemáticas estrictas respecto al tiempo, el uso de la tabla de símbolos y el recorrido del AST fueron cruciales.

Se realizaron validaciones importantes:

- Consistencia de Compases: Se verifica que la suma de las duraciones de las notas dentro de un compás coincida con la cifra indicadora definida en el TIME (ej. en 4/4, las notas deben sumar 4 negras).
- Rangos Válidos: Se valida que las octavas estén dentro de un rango audible y soportado, y que los tempos sean positivos.

Para la generación de código, se recorre el AST en profundidad. Cada nodo sabe cómo imprimirse a sí mismo en el formato de destino. Por ejemplo, un nodo Note(E, 4, quarter) se traduce a la cadena correspondiente en el lenguaje objetivo (ej. e'4 en notación LilyPond o similar).

3.3 Dificultades Encontradas

Una de las principales dificultades fue el manejo de la duración relativa y la validación de compases en el frontend y backend. Calcular si un compás está "lleno" o "incompleto" requirió implementar lógica aritmética sobre las figuras musicales (donde una redonda vale 1, una blanca 0.5, etc.) y manejar errores de precisión o desbordamiento de compás.

Otra dificultad surgió en la definición de la gramática para evitar conflictos de desplazamiento/reducción (shift/reduce) en Bison, particularmente al manejar estructuras

opcionales o secuencias de notas con distintas duraciones y modificadores (como alteraciones o ligaduras).

4. Futuras Extensiones

Siendo que el lenguaje intenta aceptar un subdominio de la notación musical, existen direcciones en las que podría expandirse para mejorar la usabilidad. Una extensión prioritaria es la flexibilización del formato de salida. Por ahora se imprime en texto plano sobre terminal con un formato rústico. Lo ideal sería implementar la posibilidad de devolverlo en formato pdf, e incluso en midi.

Actualmente, el analizador sintáctico impone restricciones que impiden apilar notas en renglones separados (por ejemplo, para representar lo que tocan ambas manos al mismo tiempo). Una futura versión debería ajustar la gramática para indicar en qué momento se avanza en la escritura y por cuánto, permitiendo al usuario escribir secuencias verticales de notas. Esto amplía considerablemente la variedad de partituras que se podrían generar.

5. Conclusiones

Como conclusión del proyecto, se pudo llevar a cabo el desarrollo de un compilador funcional para el dominio musical, pasando por las tres etapas de diseño, frontend y backend. El proyecto permitió aplicar conceptos teóricos complejos, como la traducción de una sintaxis libre (texto) a una estructura con reglas matemáticas estrictas (música).

El uso de Flex y Bison demostró ser muy potente para definir la gramática musical, y la construcción del AST fue fundamental para poder realizar las validaciones de tiempo que son el núcleo de una partitura correcta. Partitex cumple con el objetivo de simplificar la escritura musical para usuarios familiarizados con entornos de texto y programación.

6. Bibliografía

Anónimo (21 mayo 2025). Figura musical. Wikipedia. Recuperado el 27 11 2025 de https://es.wikipedia.org/wiki/Figura_musical.